



Efficient Memory Management for Large Language Model Serving with PagedAttention

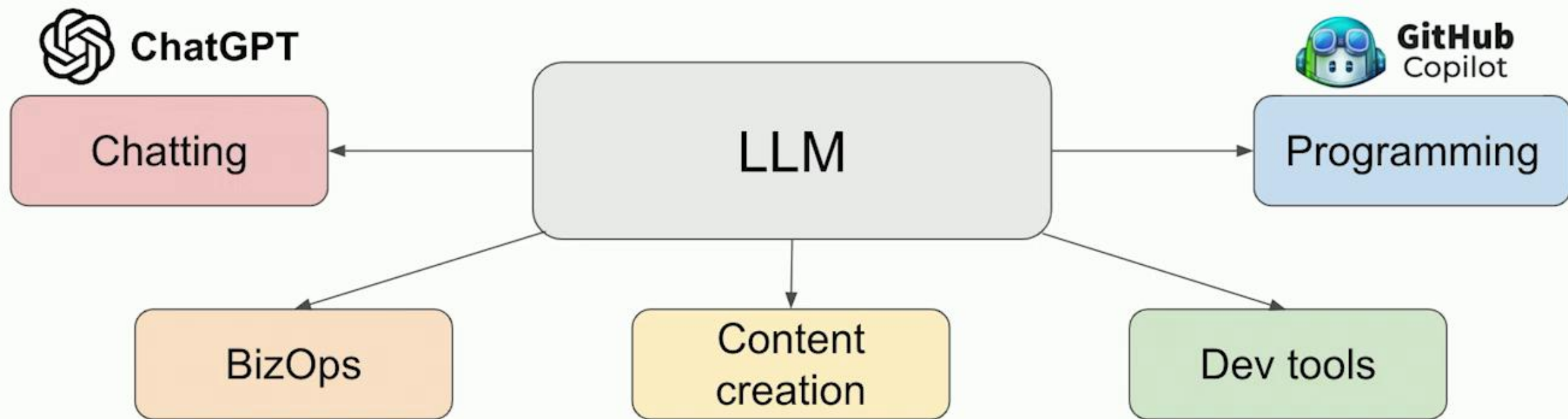
Woosuk Kwon^{1,*} **Zhuohan Li**^{1,*} Siyuan Zhuang¹ Ying Sheng^{1,2} Lianmin Zheng¹
Cody Hao Yu³ Joseph E. Gonzalez¹ Hao Zhang⁴ Ion Stoica¹

¹UC Berkeley ²Stanford University ³Independent Researcher ⁴UC San Diego

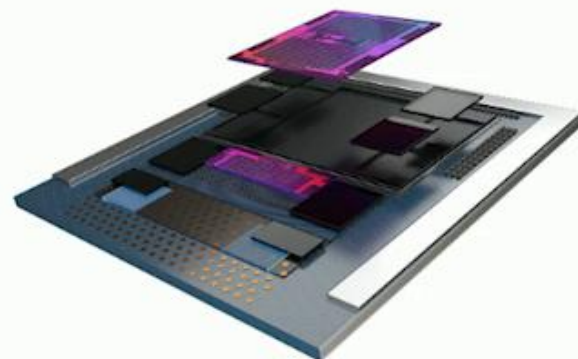
**Equal contribution*

SOSP 2023

The era of Large Language Models (LLMs)



Served by GPUs



Serving LLMs is (surprisingly) slow and expensive

- A single A100 GPU can only serve **< 1** request per second
 - Moderate size of model (13B parameters) and input
- A ton of GPUs are required for production-scale LLM services



r/LocalLLaMA • 8 days ago
by Financial_Stranger52

Join

...

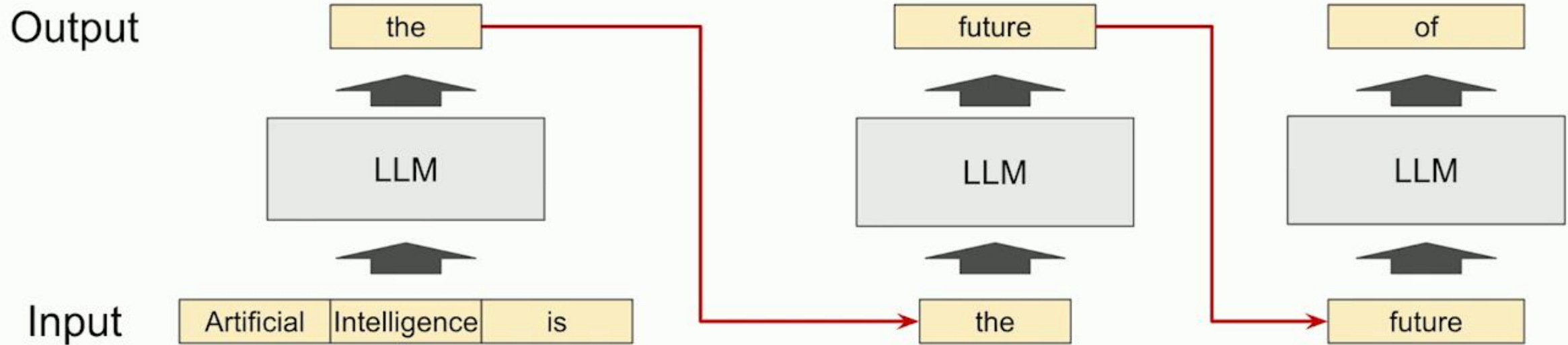
Is local LLM cheaper than ChatGPT API?

ChatGPT api only costs 0.002 dollar for 1k token. I found that LLMs like llama output only 10-20 tokens per second, which is very slow. And such machines costs over 1 dollar per hour. It seems that using api is much cheaper. Based on these observations, it seems that utilizing the ChatGPT API might be a more affordable option.



Microsoft warns of service disruptions if it can't get enough A.I. chips for its data centers

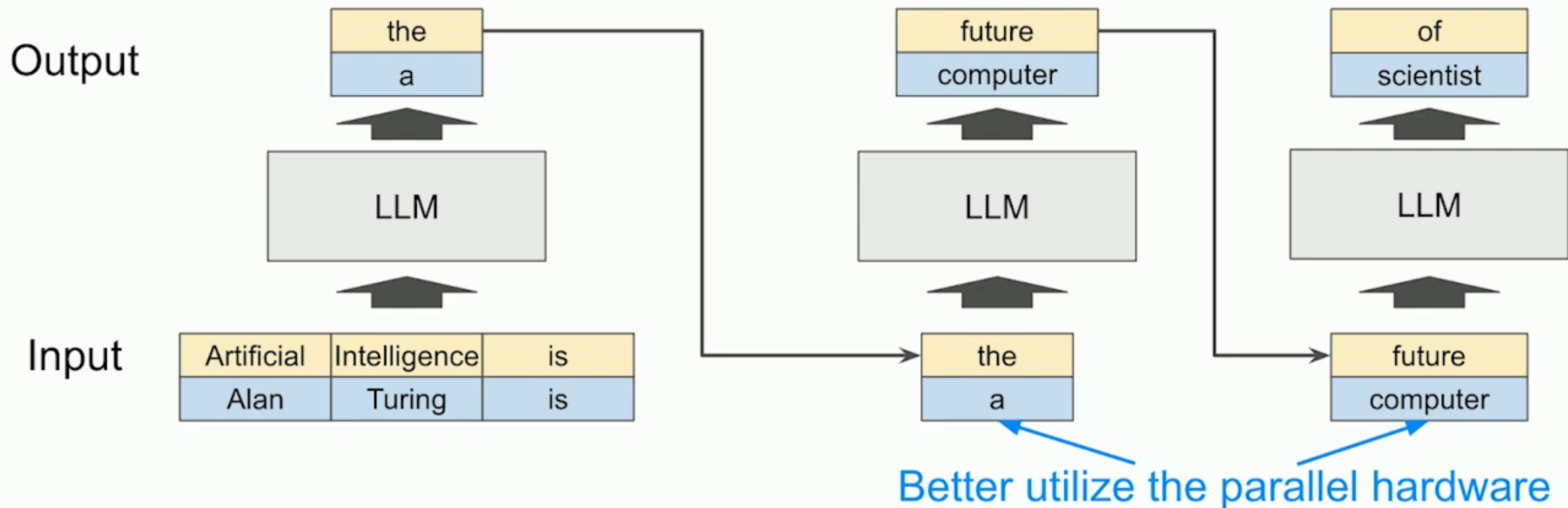
Inference process of LLMs



Why is it **slow** and **inefficient**?

- The sequential dependency between output tokens makes it difficult to fully utilize the parallelism in GPUs

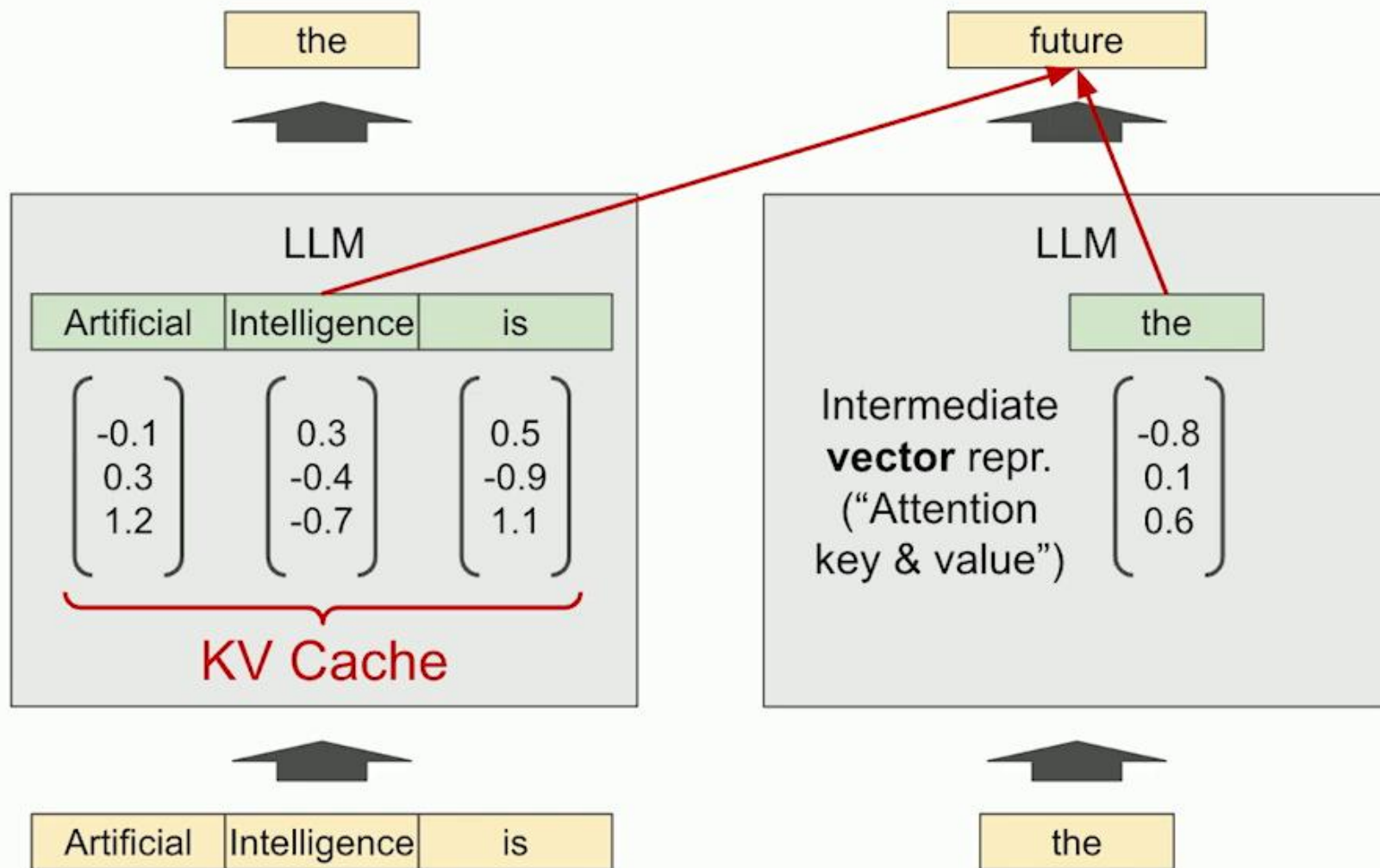
Solution: Batching multiple requests together



However, we observed that the batch size is significantly limited by the inefficient memory management for “***KV Cache.***”

(Attention) KV Cache

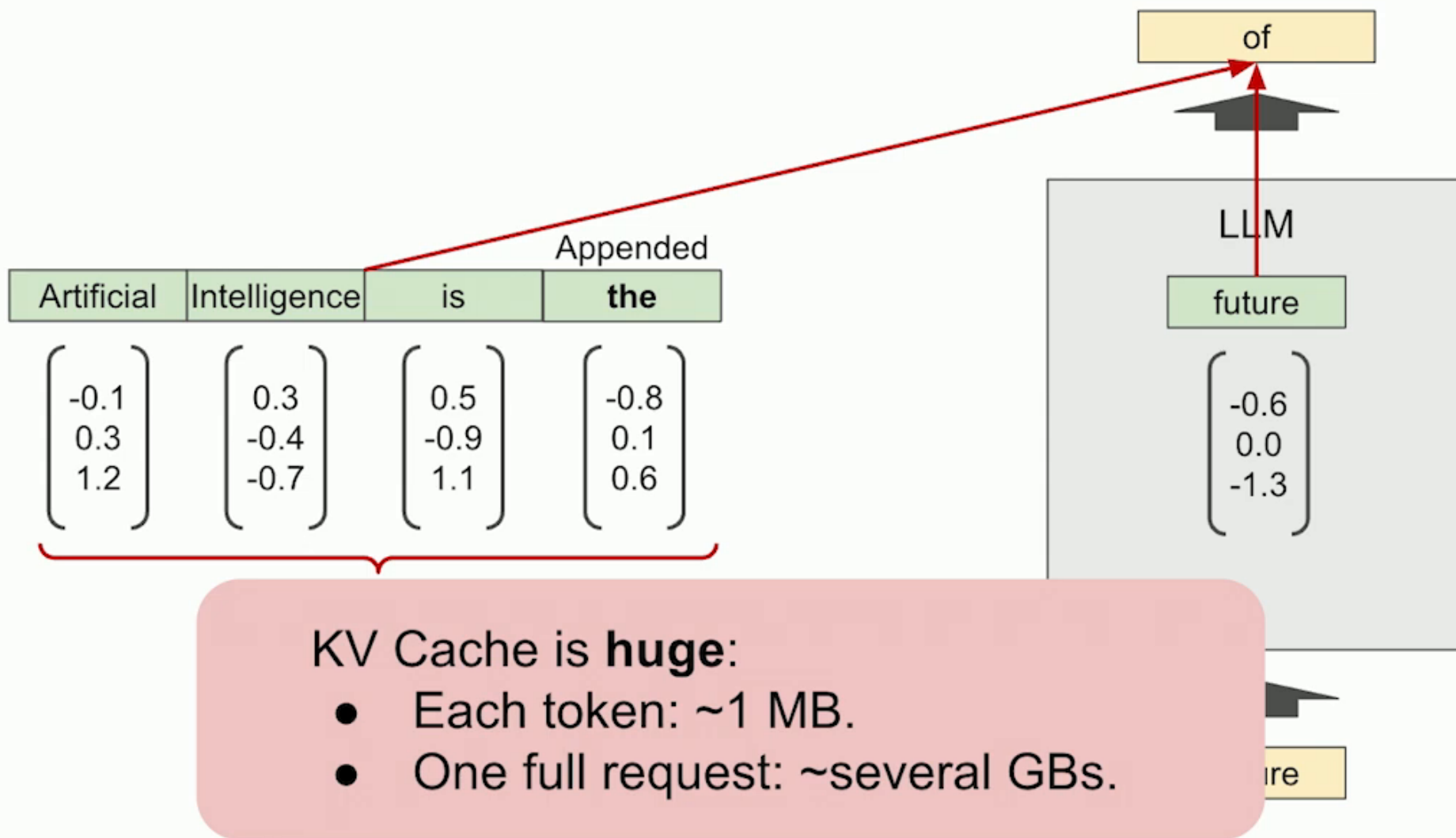
Output



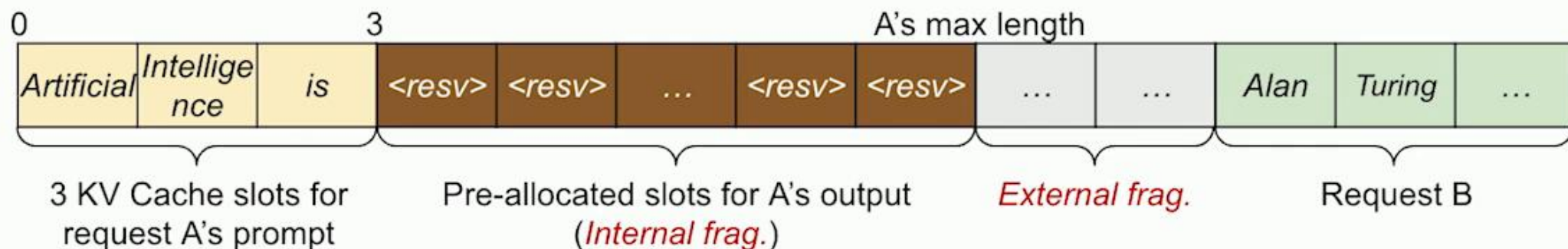
Input

KV Cache dynamically grows and shrinks

Output



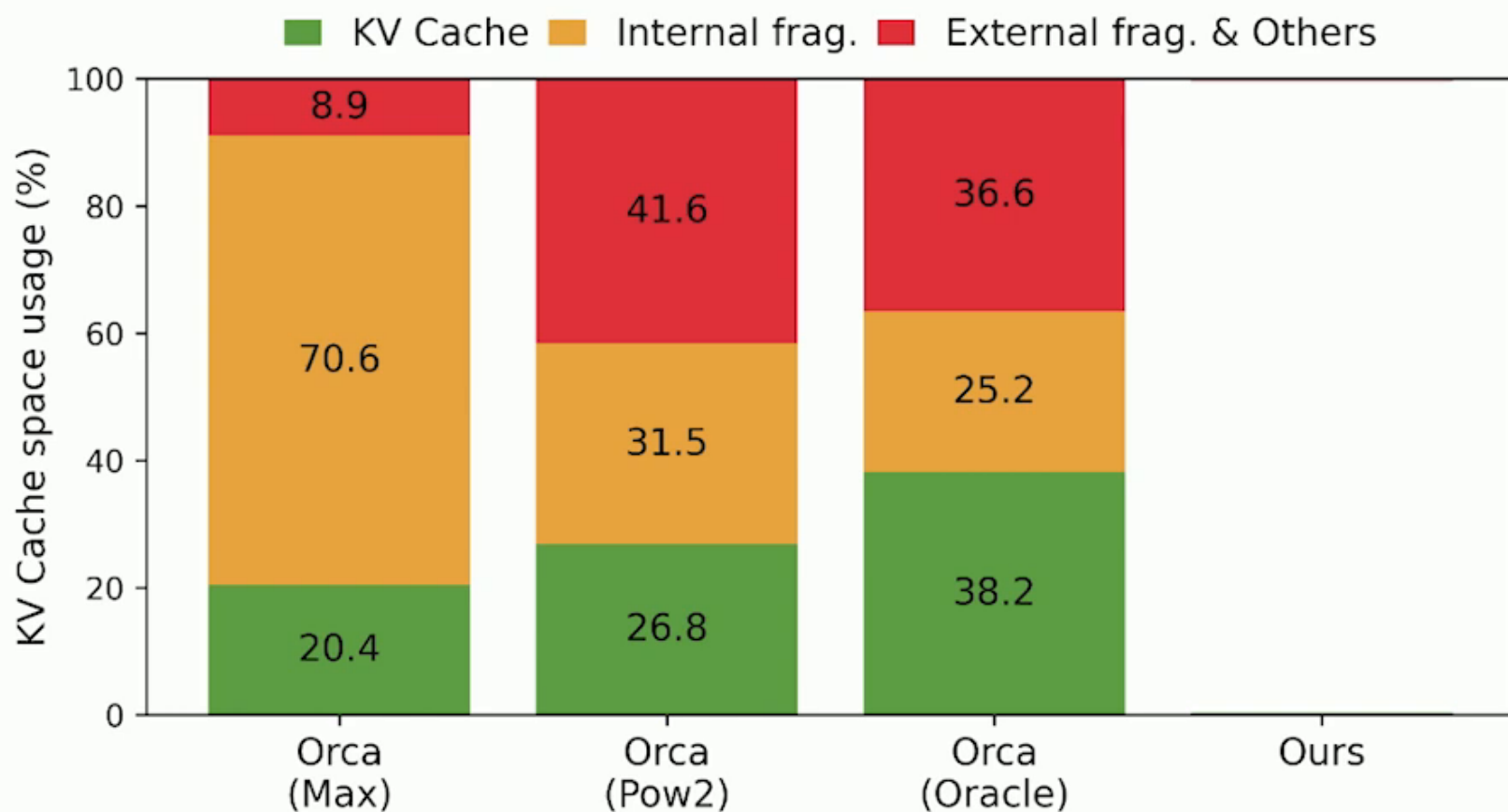
KV Cache management in previous systems



- **Pre-allocates contiguous** space of memory to the request's maximum length
 - Useful convention in traditional deep learning workloads where the input/output shapes are **static** (e.g., fast pointer arithmetic, efficient memory access)
- Results in memory fragmentation
 - **Internal fragmentation** due to the **unknown** output length.
 - **External fragmentation** due to **non-uniform** per-request max lengths.

Significant memory waste in KV Cache space

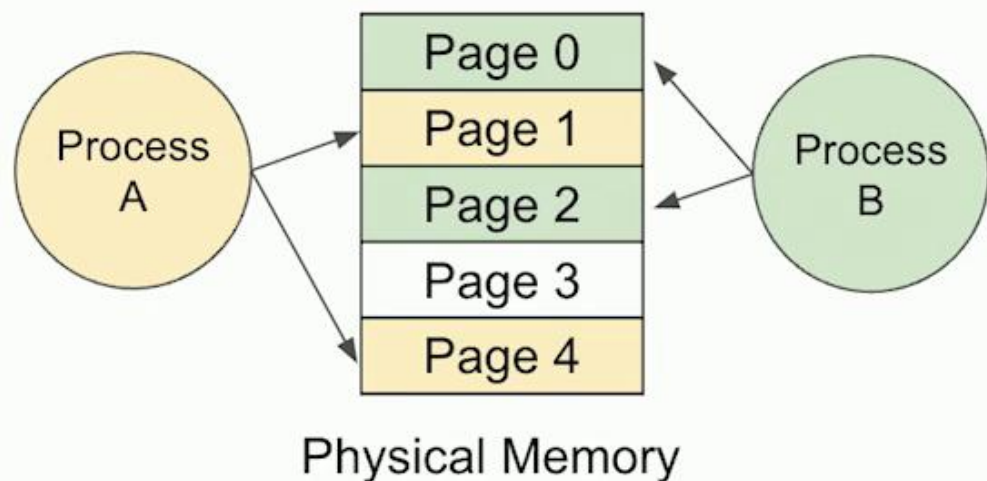
- Only **20–40%** of KV Cache space is utilized to store actual token states



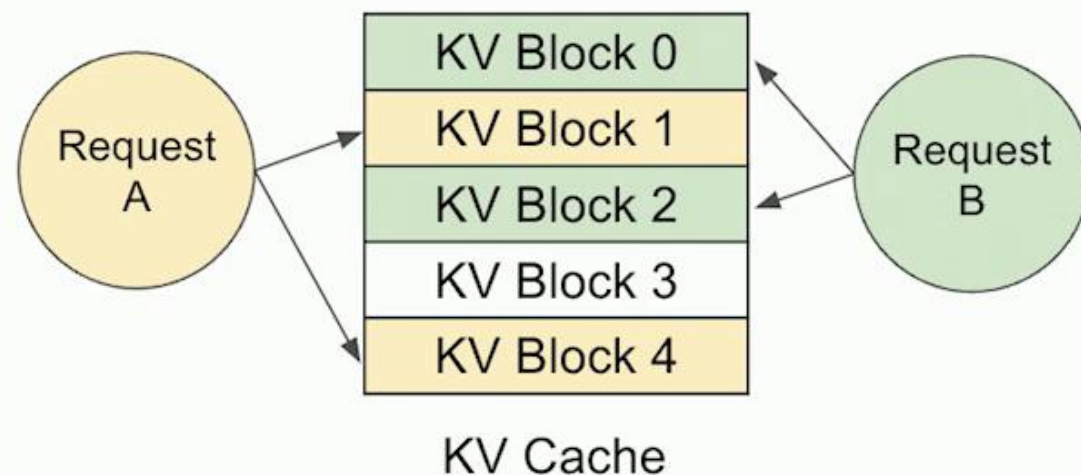
PagedAttention

- **Application-level** memory **paging** and **virtualization** for attention KV Cache

Memory management in OS

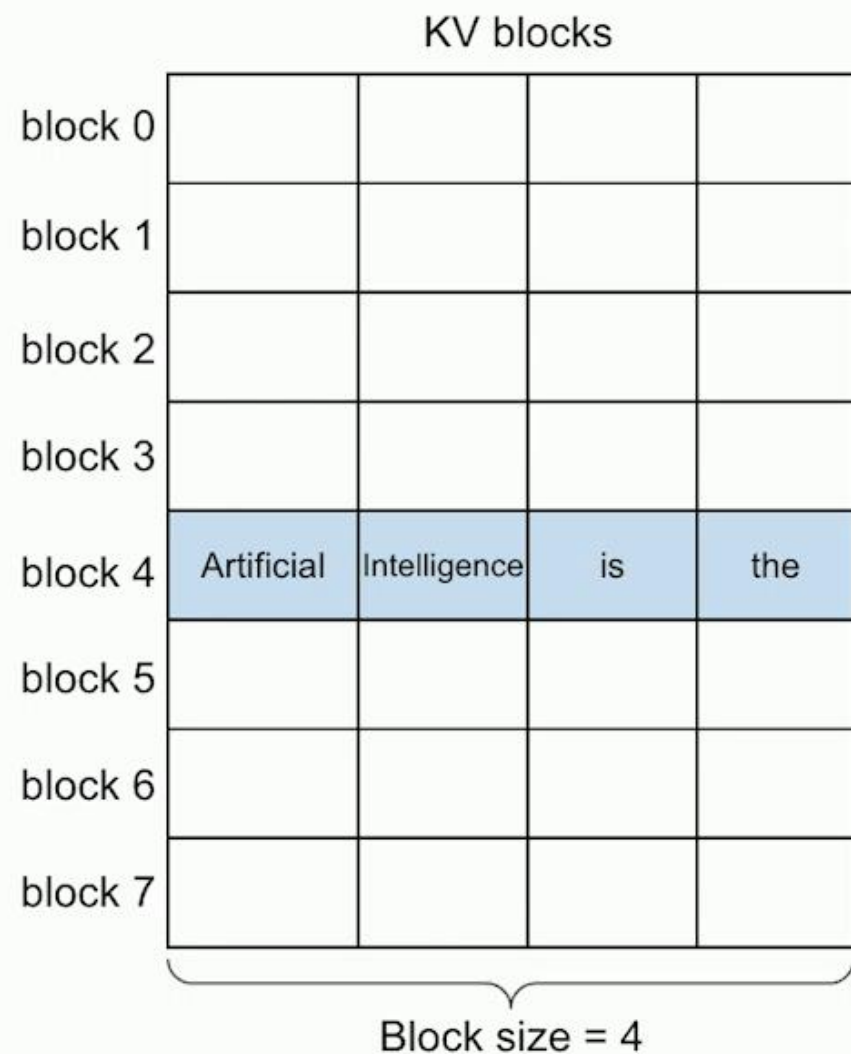


PagedAttention



Paging KV Cache space into KV blocks

- KV block is a **fixed-size** contiguous chunk of memory that can store KV token states **from left to right**



Virtualizing KV Cache



Prompt: "Alan Turing is a computer scientist"

Logical KV blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist		
block 2				
block 3				

Block table

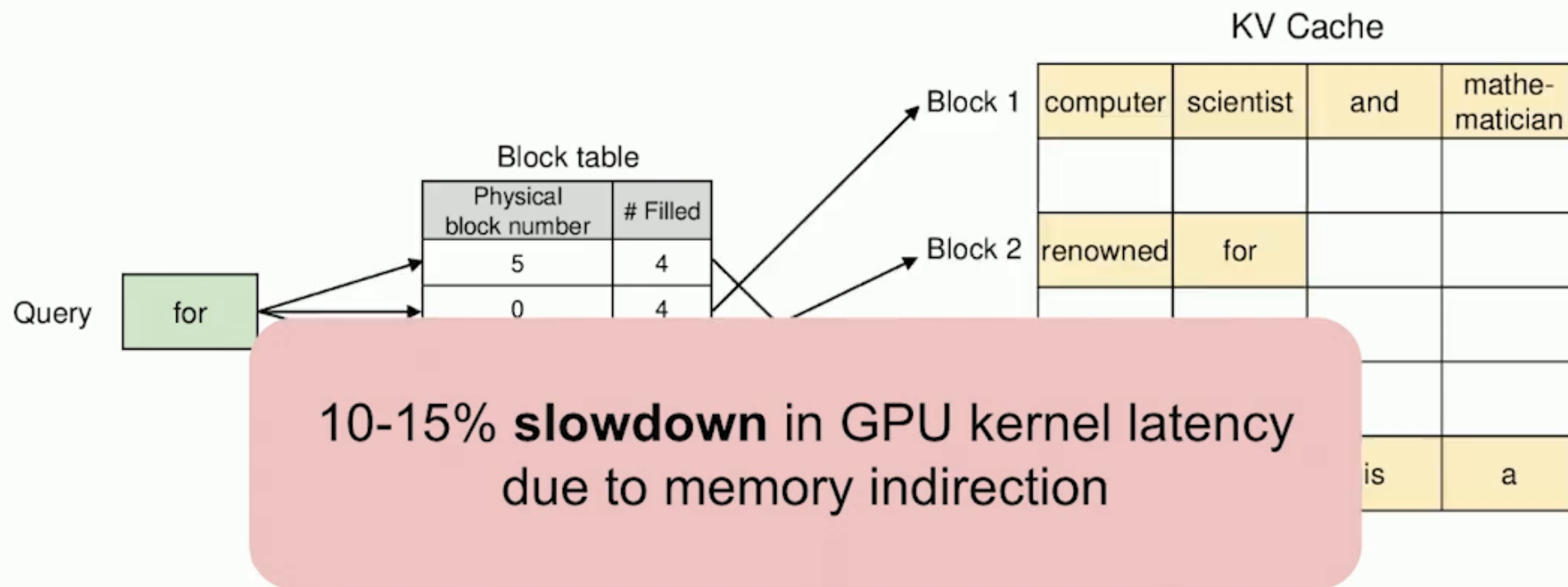
Physical block number	# Filled
7	4
1	2
—	—
—	—

Physical KV blocks

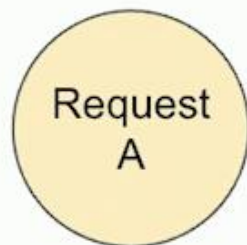
block 0				
block 1	computer	scientist		
block 2				
block 3				
block 4				
block 5				
block 6				
block 7	Alan	Turing	is	a

Attention mechanism with virtualized KV Cache

1. Fetch non-contiguous KV blocks using the block table
2. Apply attention operation on the fly



Memory management with PagedAttention



Prompt: "Alan Turing is a computer scientist"
Completion: "and mathematician renowned"

Logical KV blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist	and	mathem tician
block 2	renowned			
block 3				

Block table

Physical block number	# Filled
7	4
1	4
5	1
—	—

Physical KV blocks

block 0				
block 1	computer	scientist	and	mathem atician
block 2				
block 3				
block 4				
block 5	renowned			
block 6				
block 7	Alan	Turing	is	a

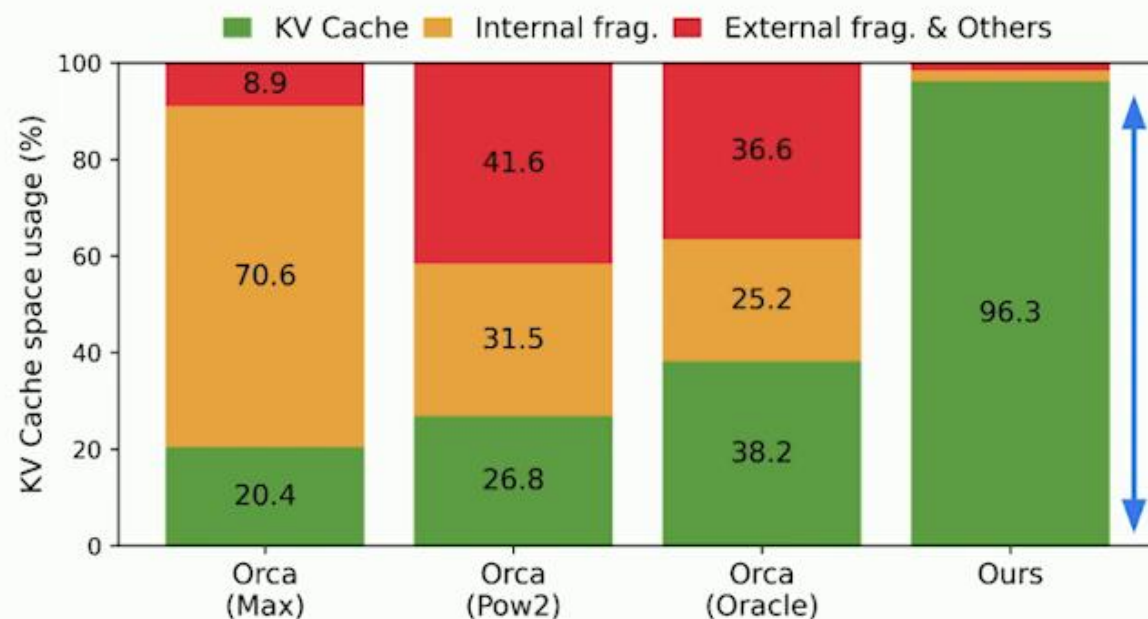
Allocated on demand

Memory efficiency of PagedAttention

- Minimal internal fragmentation
 - Only happens at the last block of a sequence
 - **# wasted tokens / seq < block size**
 - Sequence: $O(100) - O(1000)$ tokens
 - Block size: $O(10)$ tokens
- No external fragmentation

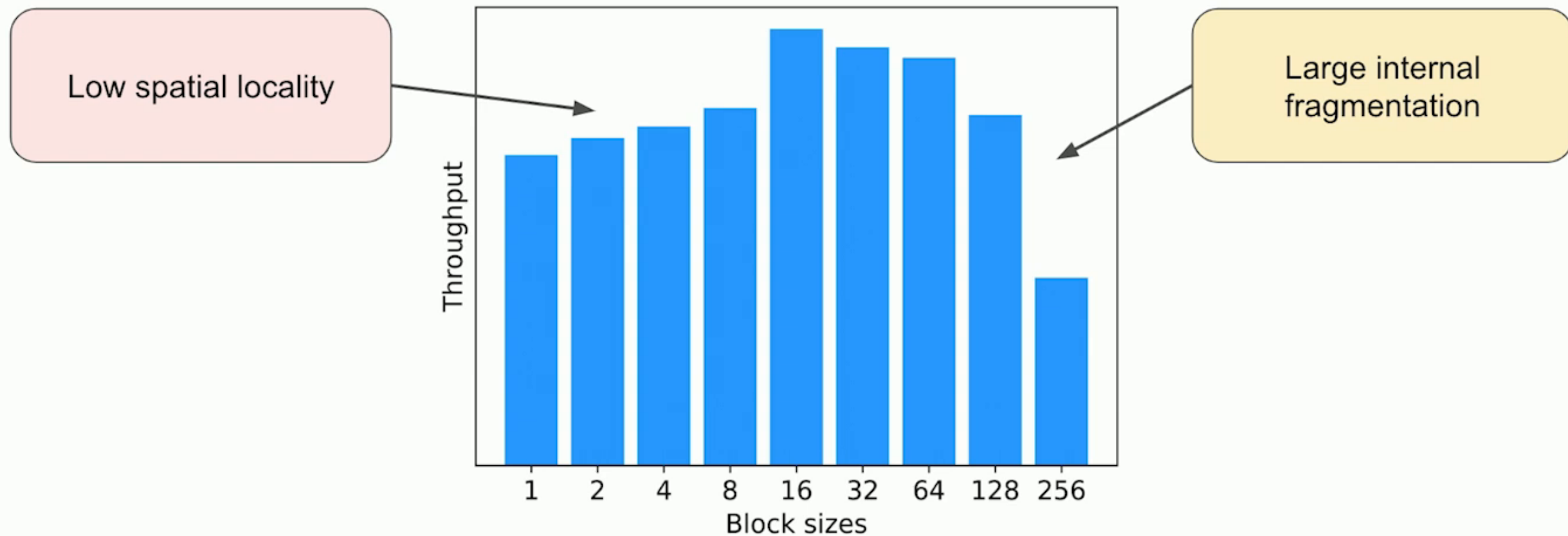
Alan	Turing	is	a
computer	scientist	and	mathemati cian
renowned			

Internal fragmentation



2.5-5x improvement

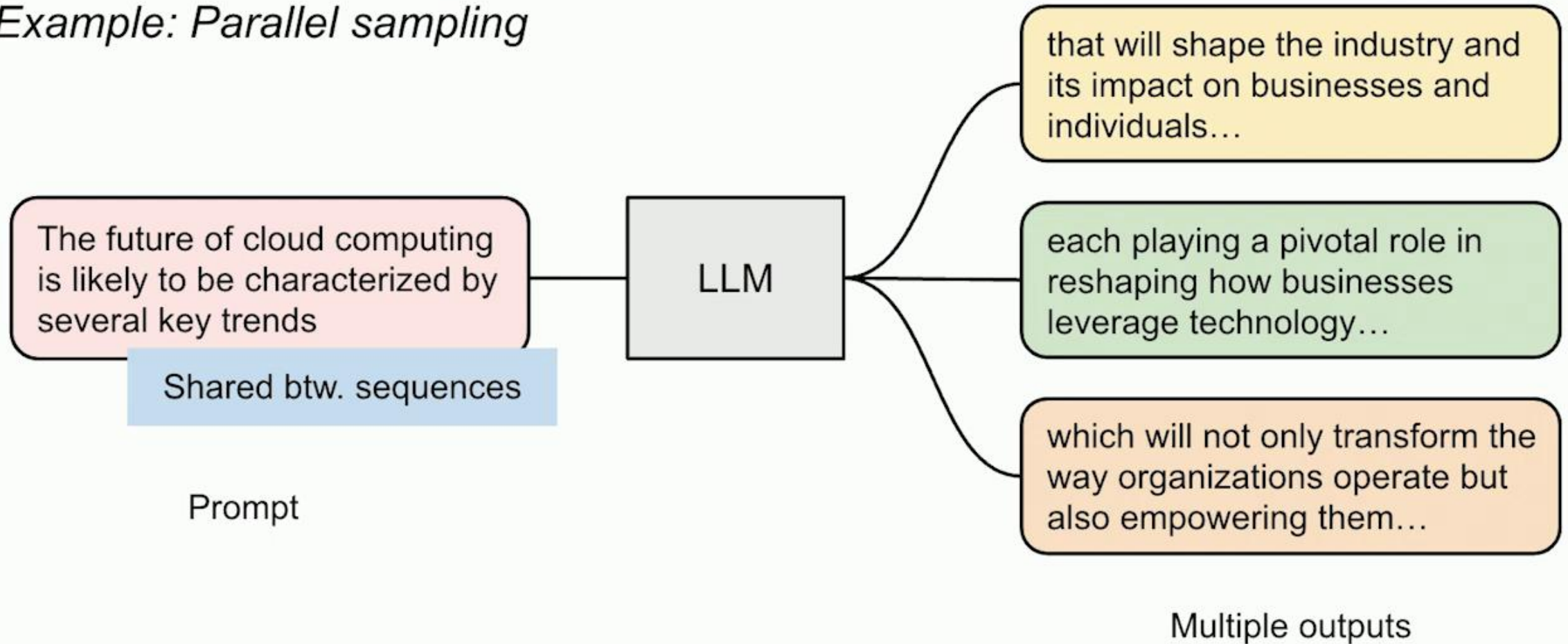
Configuring the block size



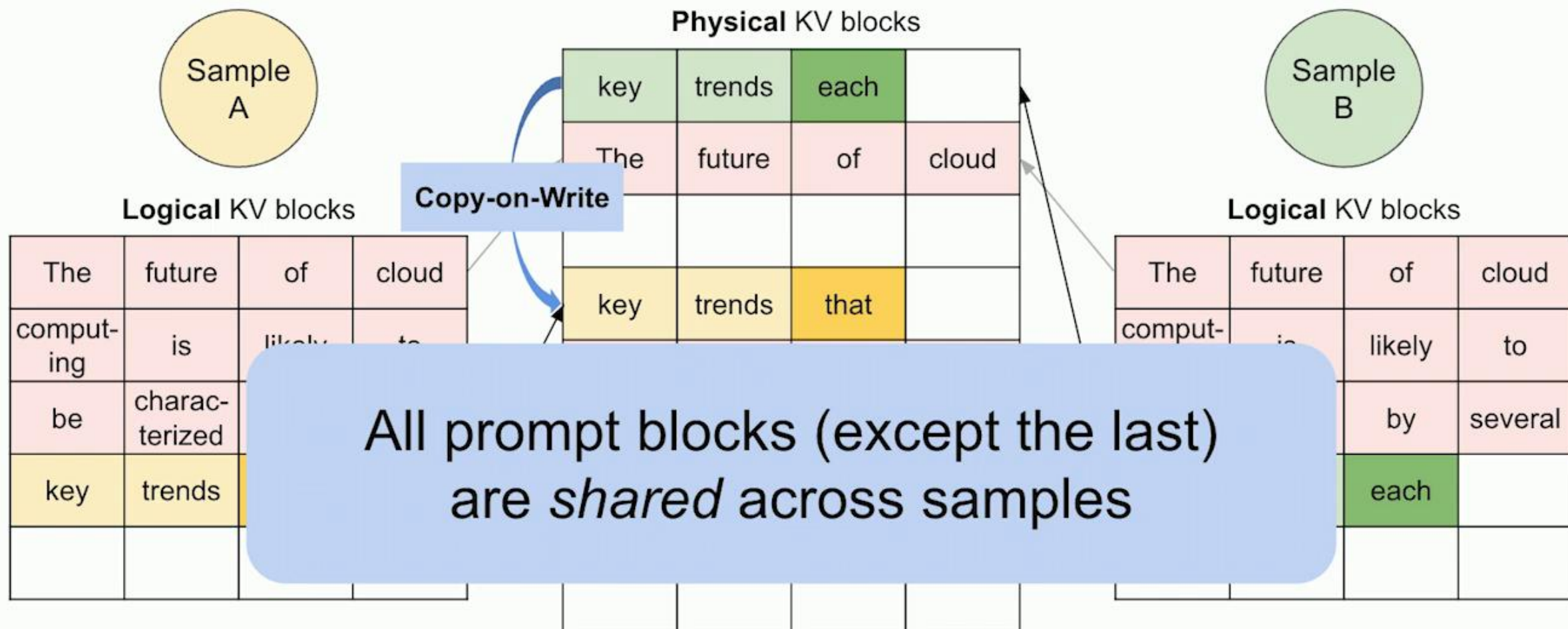
- **Block size 16** works generally well in practice

Paging enables sharing

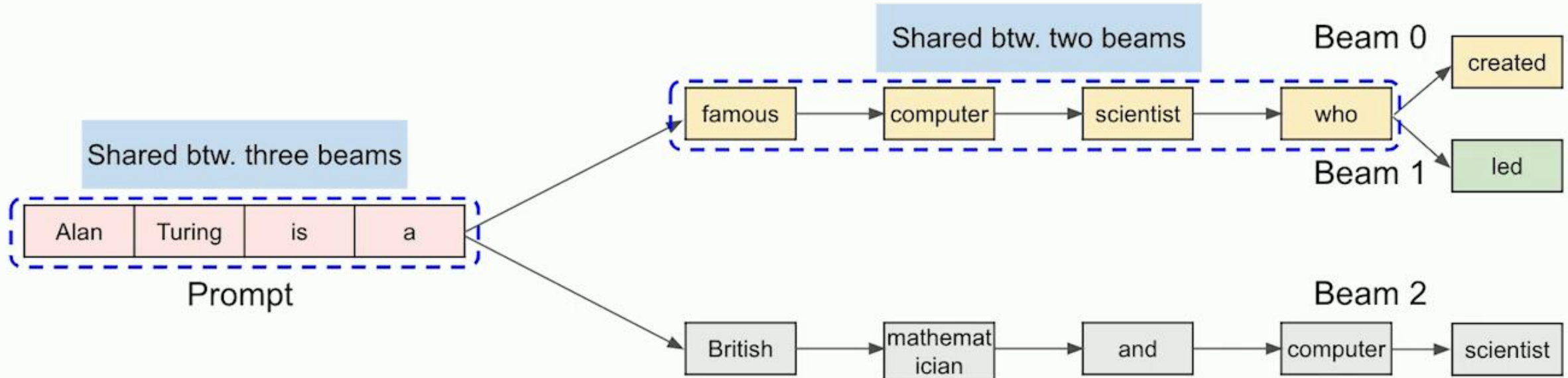
Example: Parallel sampling



Sharing KV blocks

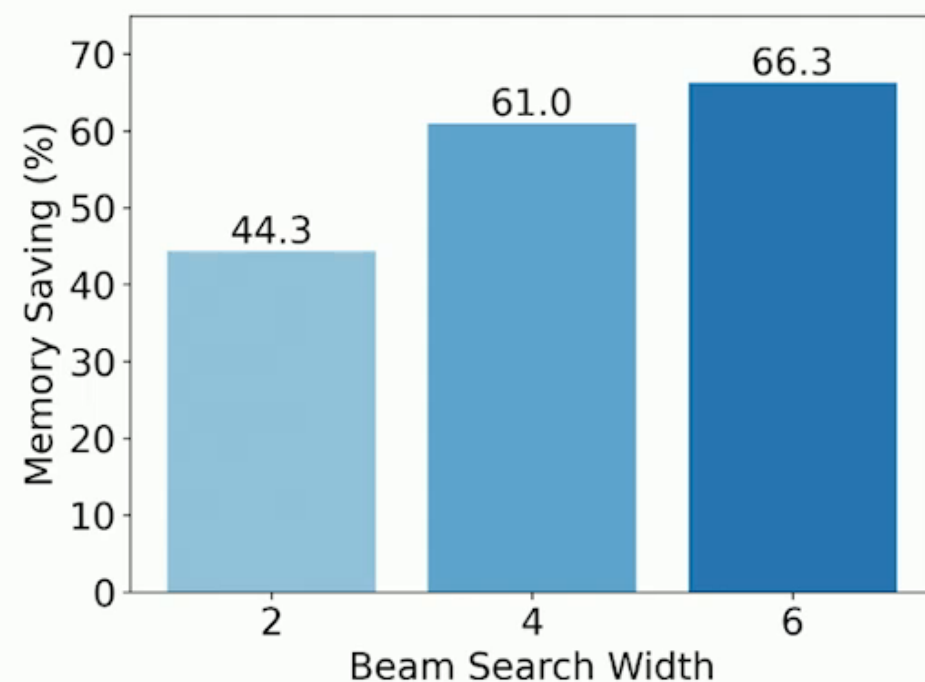
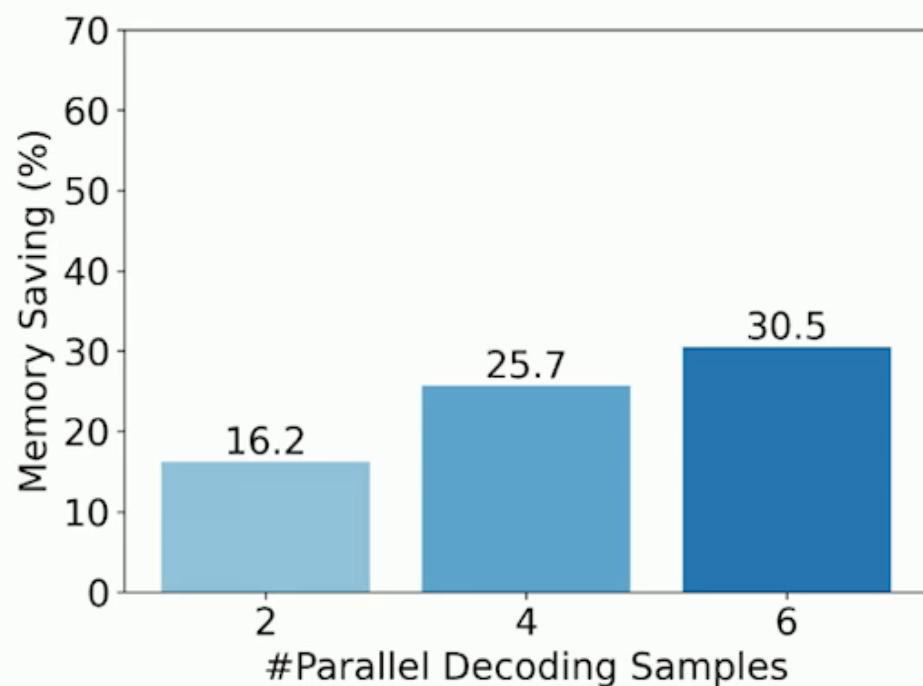


More complex sharing: beam search



- Similar to process tree (fork & kill)
- Efficiently supported by paged attention and copy-on-write mechanism

Memory saving via sharing



*Percentage = (#blocks saved by sharing) / (#total blocks without sharing)
OPT-13B on 1x A100-40G with ShareGPT dataset*

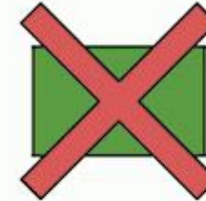
Out of KV Block Memory

Request A

Request B

Physical KV blocks

The	future	of	cloud
computer	scientist	and	mathe- matician
computing	is	likely	to
renowned	for		
Alan	Turing	is	a



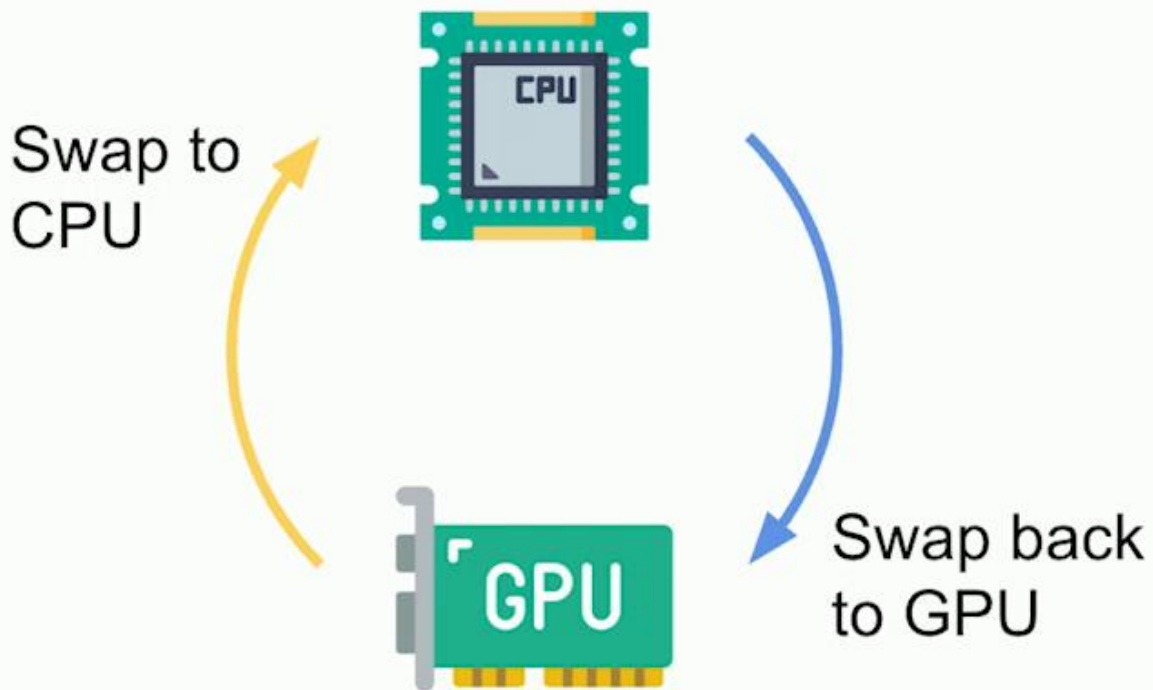
Cannot allocate a
new physical block
for Request B



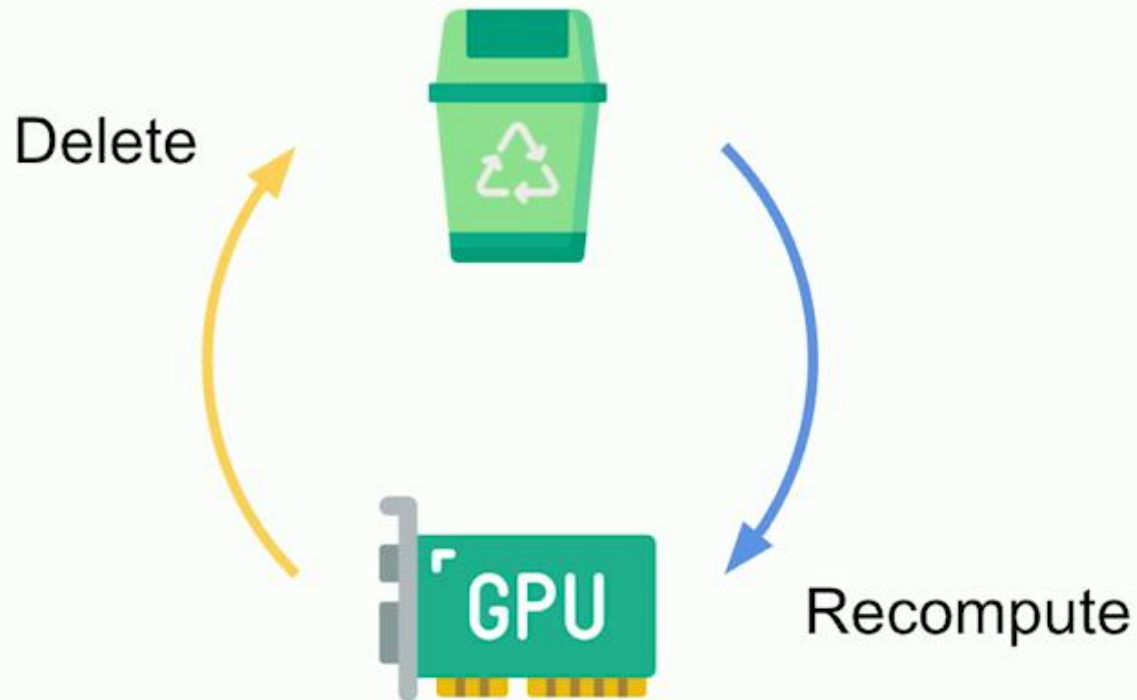
Request Preemption & Recovery

Goal: Free some requests' KV cache to let others run first.

Option 1: Swapping



Option 2: Recomputation



Notes on Preemption & Recovery

Swap/recompute **the whole request**, since all previous tokens are required every step.

Swapping: smaller block sizes $\rightarrow$ higher overhead due to small data transfers.

Recomputation: surprisingly fast since all token's KV cache can be computed in parallel.

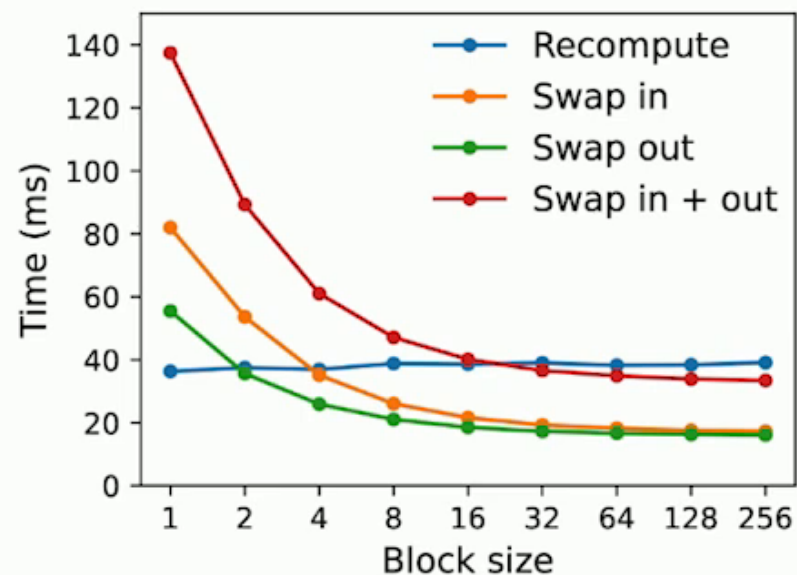


Figure: Swap/Recomputation latency of 256 tokens.

Our strategy: Use recomputation when possible with FCFS policy

Comparison with Operating System Virtual Memory

Analogies

OS pages $\leftrightarrow$ KV blocks

- Reduce memory fragmentation

Shared pages across processes

$\leftrightarrow$ Shared KV blocks across samples

- Reduce memory waste via sharing

Differences

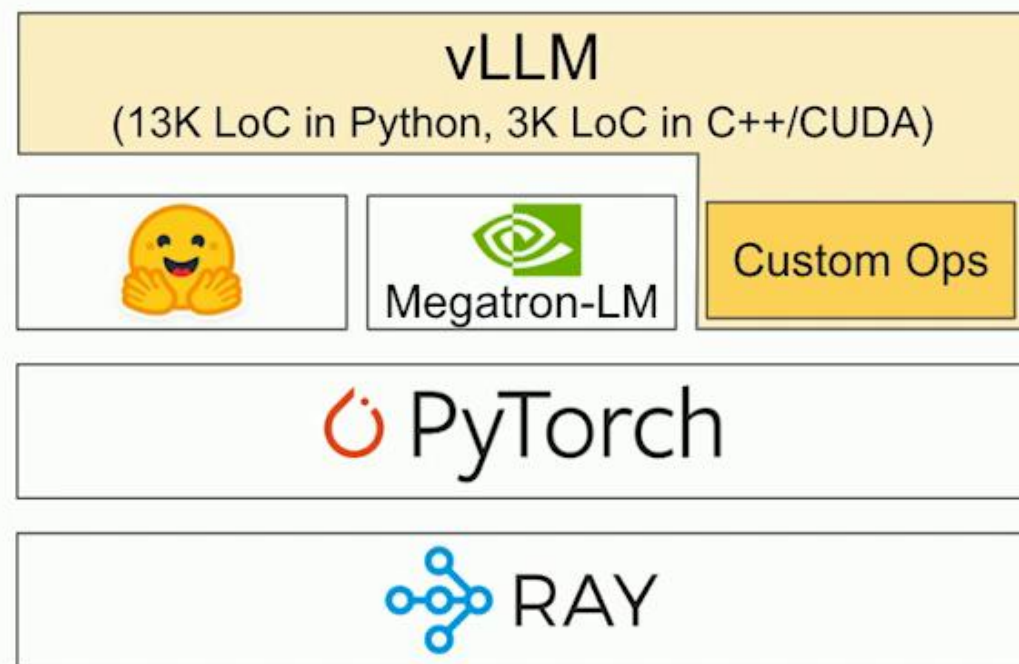
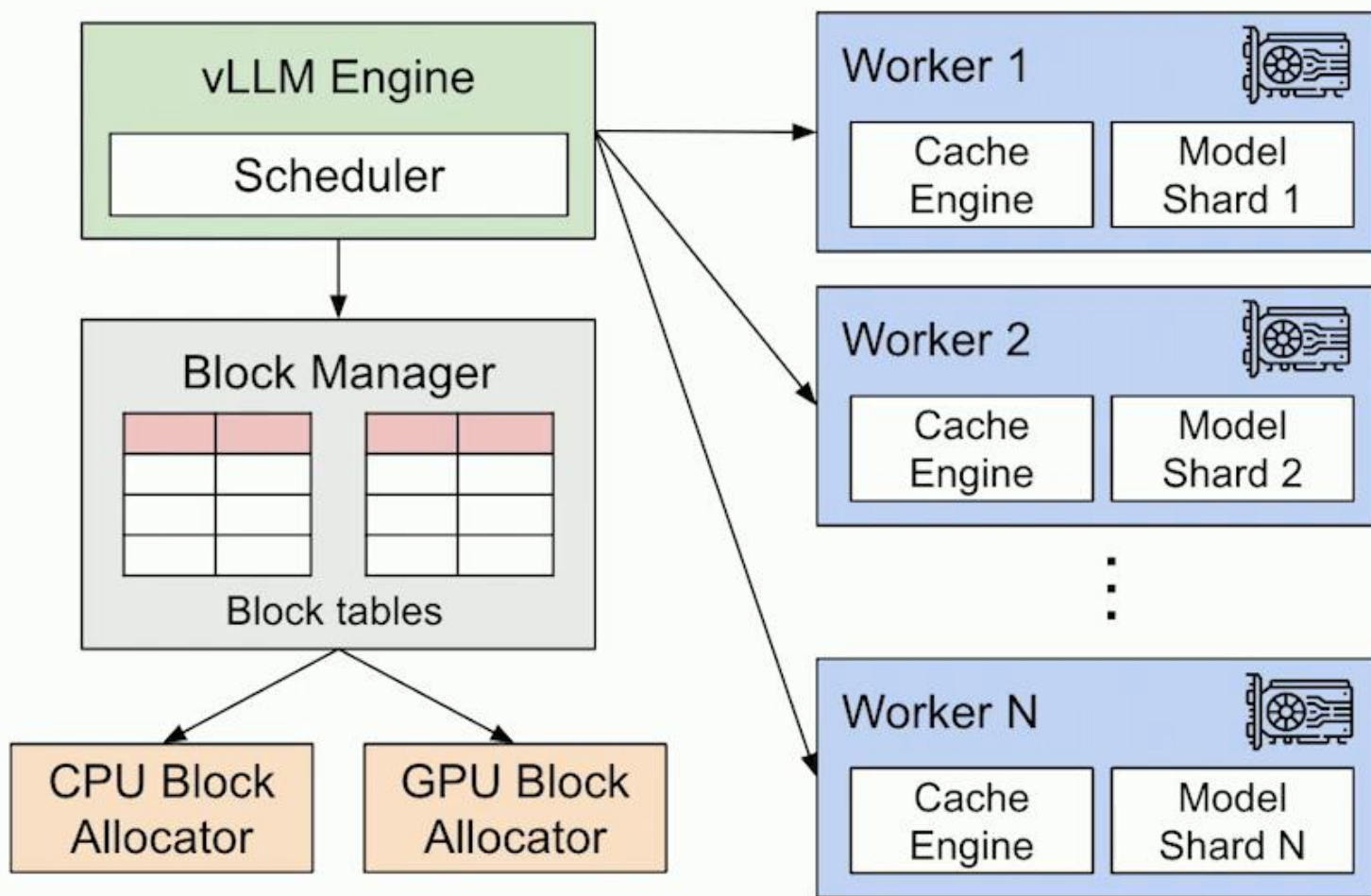
Single-level block table

- Block table is tiny compared to the actual data.

Preemption & Recovery

- Request-level preemption
- Recomputation-based recovery

vLLM Distributed System Architecture & Implementation



Evaluation – Settings

Metric: Serving throughput

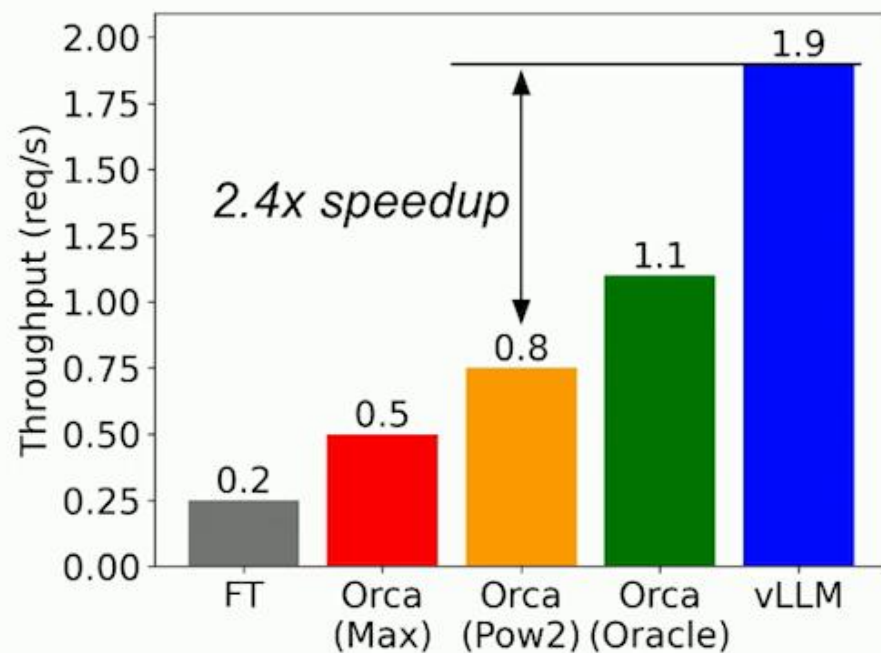
Input/Output Length Distribution

- Alpaca dataset (instruction-following)
- ShareGPT dataset (conversation)

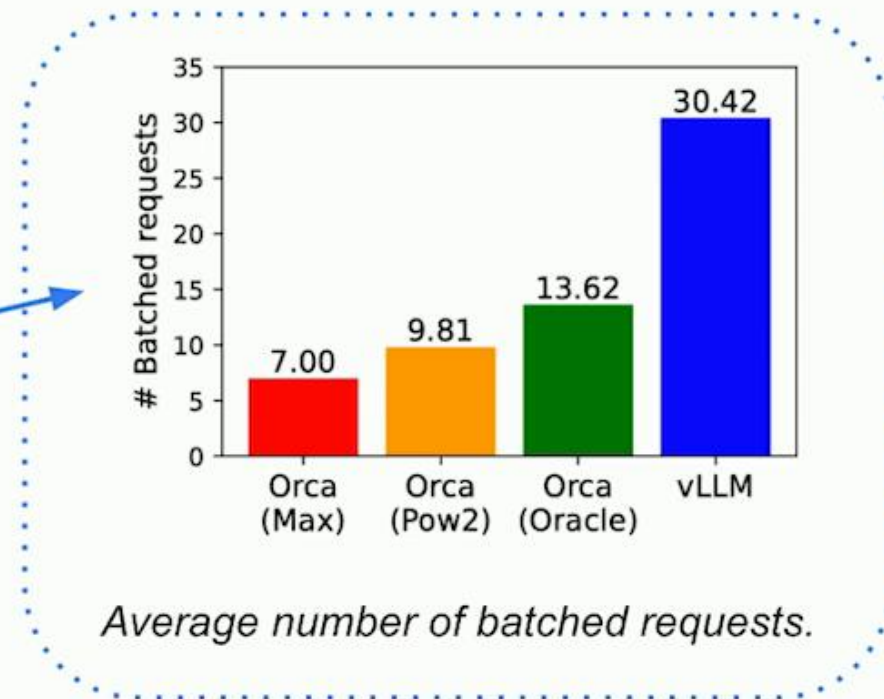
Baselines

- NVIDIA FasterTransformer (FT)
- Orca
 - Oracle: No over-reserve and know exact output lengths.
 - Pow2: Over-reserve the space for outputs by at most 2x.
 - Max: Over-reserve to the maximum possible output length.

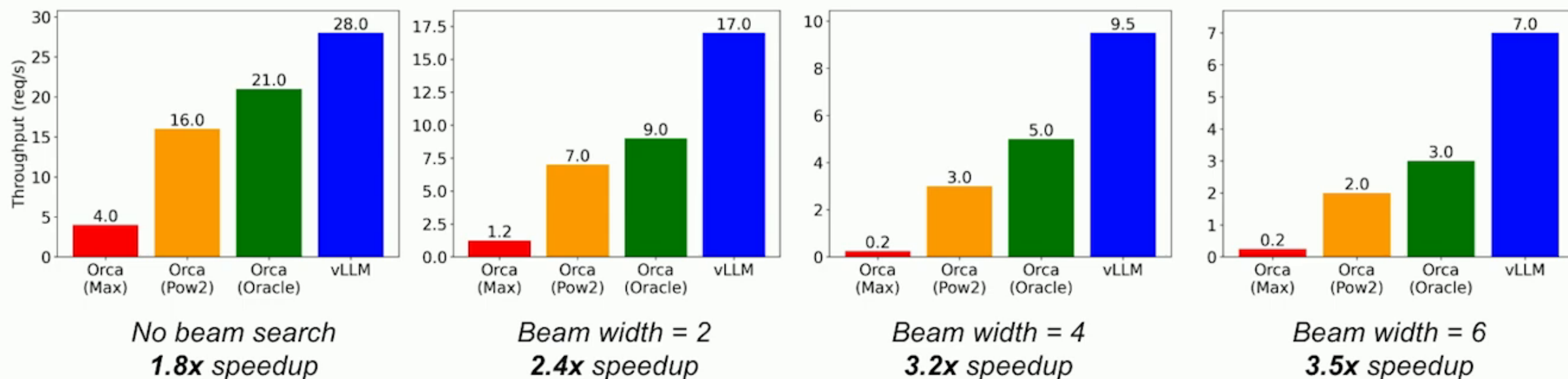
Throughput – Greedy Decoding



OPT-13B on 1xA100 40G with ShareGPT trace.



Throughput – Beam Search



OPT-13B on 1xA100 40G with Alpaca trace.
Speedup: vLLM v.s. Orca(Pow2)

vLLM Open-Source Adoption



github.com/vllm-project/vllm



```
$ pip install vllm
```

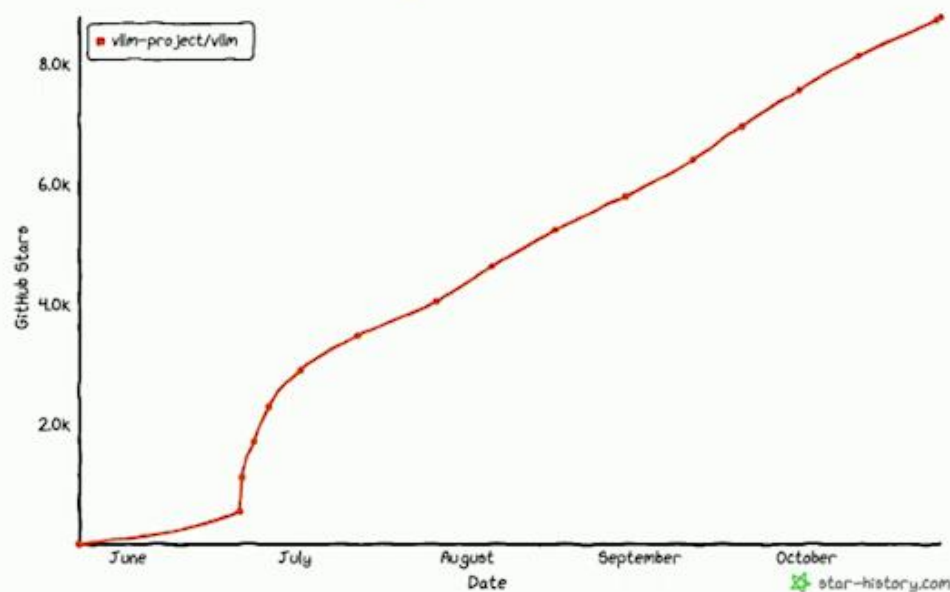


8.8K Stars



95+ Contributors

Star History



Open-Source Projects



lm-sys/FastChat



allenai/open-instruct

Companies



anyscale



C3.ai



databricks



Google Cloud



Lepton AI



Meta



Microsoft



Modal



NVIDIA



Replicate

scale



Twelve Labs

Uber

...



- Improve memory efficiency by **2.5x–5x** by **reducing fragmentation** and **enabling sharing** with *PagedAttention*
- Outperform SOTA by **up to 4x** in terms of **serving throughput**
- A vibrant open-source project that gets widely adopted



<https://github.com/vllm-project/vllm>



<https://vllm.ai>



<https://arxiv.org/abs/2309.06180>



<https://discord.gg/jz7wjKhh6g>